

Random Forest Hyperparameters

Contents

1. Overview	2
2. The three groups	3
3. The forest-level hyperparameters	3
3.1 The four settings	3
3.2 How many columns <code>max_features</code> means	4
3.3 Trying the settings on a demo dataset	4
4. The tree-level hyperparameters	6
5. The general hyperparameters	7
6. The random forest regressor	8
7. Summary	9
Sources	9
8. Key terms	9

Where this fits: Pipeline map steps 8 (Model), 10 (Tune). See the [Course map](#).



- **Builds on:** Overfitting ([Note 91](#)); Decision trees ([Note 100](#)); Feature importance ([Note 100](#)); Bagging ([Note 107](#)); OOB score ([Note 107](#)); Grid and random search ([Note 107](#)).
- **Leads to:** Grid and random search ([Note 112](#)); AdaBoost ([Note 115](#)); Balanced random forest ([Note 133](#)); Optuna ([Note 134](#)); Improving a neural network ([Note 1021](#)).
- **Compare with:** Bagging ([Note 107](#)); Dropout ([Note 1024](#)).

1. Overview

Key point: A random forest's hyperparameters fall into three groups: four that shape the forest, the usual decision tree settings for each tree, and a few general ones. Only the first group is new.

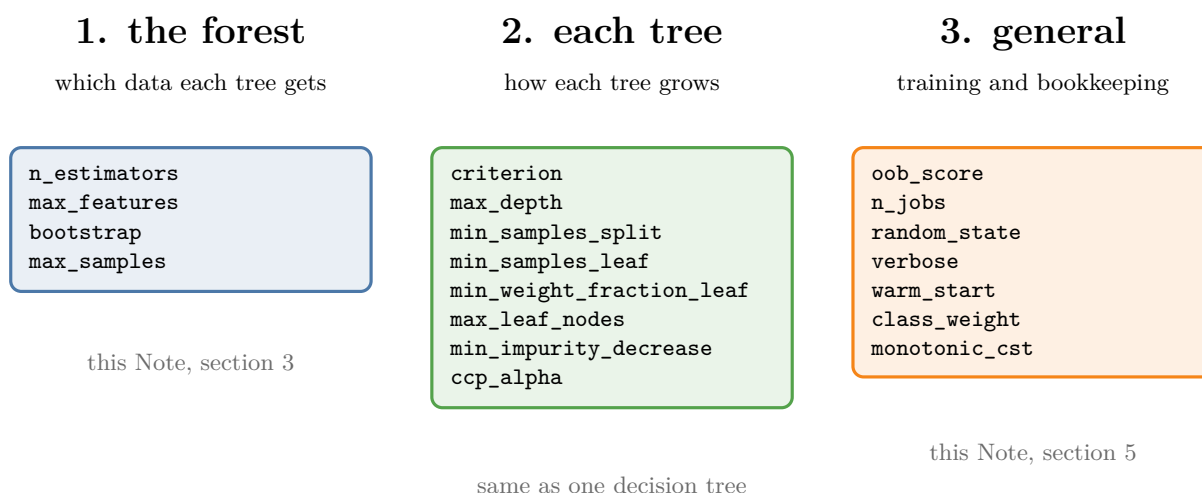


Figure 1: The hyperparameters of RandomForestClassifier, in three groups

A random forest (the [random forest introduction Note](#)) is a very flexible algorithm with many hyperparameters. Figure 1 sorts them into three groups. `RandomForestClassifier` and `RandomForestRegressor` have nearly the same settings, so learning the classifier's covers the regressor too (section 6).

The Notebook (`notebook.ipynb`) runs every experiment. The Dash app `app.py` has a control for each of the four forest-level settings and redraws the decision surface with its test accuracy.

2. The three groups

Key point: Forest settings decide which data each tree gets; tree settings decide how each tree grows; general settings handle training and bookkeeping.

1. **Forest-level** (section 3): `n_estimators`, `max_features`, `bootstrap`, `max_samples`. These tune the forest itself.
2. **Tree-level** (section 4): `criterion`, `max_depth`, `min_samples_split` and the rest. A forest is built from decision trees, and these are the settings of every one of them.
3. **General** (section 5): `n_jobs`, `random_state`, `verbose` and others found in many scikit-learn algorithms.

3. The forest-level hyperparameters

Key point: `n_estimators` sets the number of trees, `max_samples` the rows per tree, `max_features` the columns considered at each split, and `bootstrap` whether rows are drawn with replacement.

3.1 The four settings

Key point: Two settings size the forest and its samples; two decide how rows and columns are drawn.

Setting	Default	Meaning
<code>n_estimators</code>	100	the number of trees in the forest
<code>max_features</code>	"sqrt" (classifier), 1.0 (regressor)	how many columns each split considers
<code>bootstrap</code>	True	draw each tree's rows with replacement (True) or give every tree the whole training set (False)
<code>max_samples</code>	None (as many rows as the training set)	how many rows each tree gets; only used when <code>bootstrap=True</code>

`max_features` and `max_samples` accept a whole number (a count) or a decimal (a share), as in `BaggingClassifier` (the [bagging classifier Note](#), section 3.2).

3.2 How many columns `max_features` means

Key point: “sqrt” takes the square root of the number of columns, “log2” its base-2 logarithm, a decimal a share, and `None` all of them; each rounds down.

At every split, a tree draws `max_features` columns at random and picks the best split among them (node-level sampling: the [bagging vs random forest Note](#), section 3.2). With p columns:

1. **In words:** turn the setting into a number of columns, then round down (but never below 1).
2. **Formula:**

$$\text{"sqrt"} : \lfloor \sqrt{p} \rfloor \quad \text{"log2"} : \lfloor \log_2 p \rfloor \quad \text{decimal } f : \lfloor f \cdot p \rfloor \quad \text{None} : p$$

where $\lfloor \cdot \rfloor$ means rounding down.

3. **Example:** with $p = 100$ columns: “sqrt” gives $\sqrt{100} = 10$; “log2” gives $\log_2 100 = 6.64$, so **6**; 0.2 gives 20; `None` gives all 100. The Notebook confirms each one.

With the 2 columns of our demo data, “sqrt” gives $\lfloor 1.41 \rfloor = 1$: each split may look at only one randomly chosen column.

Extra: Older code and documentation also list `max_features="auto"`, which meant “sqrt” for the classifier and all the columns for the regressor. It was removed in scikit-learn 1.3 and now raises an error; the classifier’s default became “sqrt” in 1.1 and the regressor’s 1.0 (all columns). (scikit-learn API docs)

3.3 Trying the settings on a demo dataset

Key point: More trees smooth the boundary up to a point; very few rows per tree hurt; the other settings change little on 2-column data.

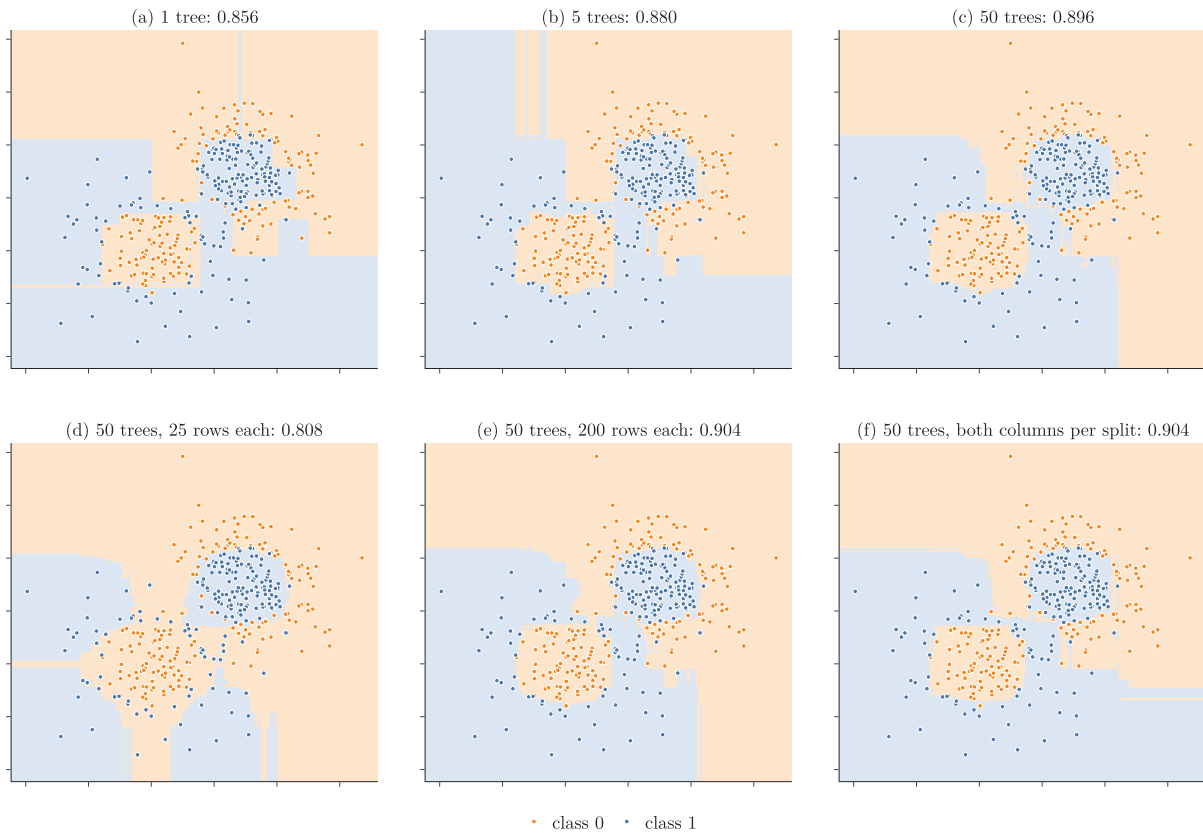


Figure 2: Random forests on the demo data with different forest-level settings; titles give the test accuracy

The demo data has 500 points with 2 columns and 2 classes: two rings of points, each around a blob of the other class. We train on 375 points and test on 125. Figure 2 shows six forests; the Dash app lets us try any combination.

n_estimators. One tree scores 0.856, 5 trees 0.880, 10 trees 0.888 and 50 trees 0.896, and then it stops improving (100 and 200 trees: 0.896).

- With **few trees** the boundary is erratic, with odd strips and corners that show overfitting (Figure 2a, b).
- With **more trees** these areas smooth out (Figure 2c).
- Each extra tree costs training time, so beyond the point where the score stops rising, more trees only add cost.

max_samples (with 50 trees). Very few rows per tree hurt: 25 rows score 0.808 (Figure 2d), 50 and 100 rows 0.880. With 200 or 280 rows (53% and 75% of 375) the score reaches 0.904 (Figure 2e); all 375 rows give 0.896. On this data the best share is 53% to 75% of the rows. The best share depends on the data: on the heart data of the [random forest tuning Note](#), smaller shares do a little better.

max_features (with 50 trees). One column per split (the default here) scores 0.896, both columns 0.904 (Figure 2f). Even with one column the forest does well, because each split gets a randomly chosen column, so both columns are used across the tree.

bootstrap (with 50 trees). **True** scores 0.896, **False** 0.880. There is little difference; **True**, the default, is the usual choice.

Extra: With **bootstrap=False**, every tree is trained on the whole training set: the rows are not drawn without replacement, they are not sampled at all. So **max_samples** cannot be set (scikit-learn raises a **ValueError**). If, on top of that, **max_features=None**, the only randomness left is tie-breaking between equally good splits (scikit-learn API docs, **DecisionTreeClassifier**, **random_state**), and the trees come out nearly identical (45 or 46 leaves each in the Notebook): a forest of copies of one tree.

4. The tree-level hyperparameters

Key point: Every tree in the forest takes the usual decision tree settings; they work exactly as for a single tree.

These settings are applied to each tree in the forest. Each is explained, with its effect on overfitting, in the [decision tree hyperparameters Note](#):

Setting	Default	What it controls
criterion	"gini"	how to measure split quality: "gini", "entropy" or "log_loss"
max_depth	None (fully grown)	the deepest a tree may grow
min_samples_split	2	the fewest rows a node needs before it may split
min_samples_leaf	1	the fewest rows allowed in a leaf
min_weight_fraction_leaf	0.0	the same as min_samples_leaf , as a share of all the rows
max_leaf_nodes	None	the most leaves a tree may have
min_impurity_decrease	0.0	the smallest impurity decrease worth a split
ccp_alpha	0.0 (no pruning)	cost-complexity pruning (Extra below)

Most are pruning settings: they stop a tree before it fits every training point. In a random forest they are usually left at their defaults, because the trees are meant to be fully grown (low bias); the forest removes the variance (the [random forest and bias-variance Note](#)).

Extra: `ccp_alpha` prunes a grown tree back. Every subtree is scored by its training error plus `ccp_alpha` times its number of leaves, and the subtree with the lowest score is kept. A larger `ccp_alpha` charges more for each leaf, so the tree gets smaller. In the Notebook, with 50 trees on the demo data: `ccp_alpha=0` leaves 41.6 leaves per tree on average (test accuracy 0.896); 0.002 leaves 39.0 (0.912); 0.01 leaves 12.1 (0.904); 0.05 leaves only 3.7, and the forest underfits (0.752).

5. The general hyperparameters

Key point: `oob_score` gives a free accuracy estimate, `n_jobs` speeds training up, `random_state` makes runs repeatable, `warm_start` adds trees, `class_weight` helps imbalanced data.

Setting	Default	Meaning
<code>oob_score</code>	<code>False</code>	score the forest on its out-of-bag rows (the OOB score Note)
<code>n_jobs</code>	<code>None</code> (1 core)	train trees in parallel on several CPU cores; -1 means all
<code>random_state</code>	<code>None</code>	fixes the random draws of rows and columns, so the same settings give the same forest
<code>verbose</code>	0	print progress during training and prediction
<code>warm_start</code>	<code>False</code>	<code>True</code> keeps the trees already trained and adds new ones on the next <code>fit</code>
<code>class_weight</code>	<code>None</code>	weights for each class, for imbalanced data (as in the logistic regression hyperparameters Note)
<code>monotonic_cst</code>	<code>None</code>	forces predictions to only rise (1) or only fall (-1) as a column grows

Two of them deserve a closer look:

- **random_state.** Without it, training the same forest twice gives slightly different results, because the rows and the columns at every split are drawn at random. Fixing it, for example `random_state=42`, makes the results repeatable.
- **warm_start.** The forest can be trained in stages: train 50 trees, set `n_estimators=100`,

call `fit` again, and only the 50 new trees are trained. This is handy for finding how many trees are enough without retraining from scratch.

Python: Adding trees with `warm_start`.

```
rf = RandomForestClassifier(n_estimators=50,
                           warm_start=True, random_state=42)
rf.fit(X_train, y_train)           # 50 trees
rf.set_params(n_estimators=100)
rf.fit(X_train, y_train)           # 50 more: 100 in total
```

`set_params` changes a setting of a model after it is created. The new trees are trained on the data passed to the second `fit`.

Extra: `monotonic_cst` (added in scikit-learn 1.4) takes one value per column: 1, -1 or 0 (no constraint). For example, a model of house prices can be forced to never predict a lower price for a bigger house. It works for regression and for two-class classification, not for multi-class or multi-output problems (scikit-learn API docs).

6. The random forest regressor

Key point: Same hyperparameters, except the criterion options and the `max_features` default.

`RandomForestRegressor` has the same settings as the classifier, with two differences:

- **criterion:** "squared_error" (the default; mean squared error, the variance reduction of the [regression trees Note](#)), "absolute_error" or "poisson", instead of Gini and entropy.
- **max_features:** the default is 1.0, all the columns at every split, instead of "sqrt".

It has no `class_weight`, since there are no classes.

Extra: Older code uses `criterion="mse"` and `"mae"`; these were renamed "squared_error" and "absolute_error" and the old names were removed in scikit-learn 1.2. A fourth option, "friedman_mse", is deprecated since 1.9. The setting `min_impurity_split`, listed as deprecated in old documentation, was removed in 1.0; `min_impurity_decrease` replaces it (scikit-learn API docs).

7. Summary

Group	Settings	Usually
Forest	<code>n_estimators</code>	100 or more, until the score stops rising
	<code>max_samples</code>	50% to 75% of the rows (only with <code>bootstrap=True</code>)
	<code>max_features</code>	“sqrt” (classifier) or all columns (regressor); tune it
	<code>bootstrap</code>	<code>True</code>
Each tree	<code>criterion</code> , <code>max_depth</code> , <code>min_samples_split</code> , ...	defaults: fully grown trees
General	<code>n_jobs=-1</code> , <code>random_state</code> , <code>oob_score</code> , <code>warm_start</code> , <code>class_weight</code>	as needed

- On the demo data, accuracy rose from 0.856 (1 tree) to 0.896 (50 trees), then stayed flat.
- 25 rows per tree scored only 0.808; 200 rows (about 50%) 0.904.
- `bootstrap=False` means every tree gets all the rows; `max_samples` then cannot be set.
- The regressor differs only in its criteria and its `max_features` default.

Sources

- scikit-learn developers. API reference pages for `RandomForestClassifier`, `RandomForestRegressor` and `DecisionTreeClassifier`, versions 0.24, 1.1 and 1.9 (version notes under each parameter).

8. Key terms

Term	Meaning
Forest-level hyperparameters	The settings that shape the forest itself: <code>n_estimators</code> , <code>max_features</code> , <code>bootstrap</code> , <code>max_samples</code>
<code>ccp_alpha</code>	Cost-complexity pruning strength: the penalty per leaf when a grown tree is pruned back
<code>warm_start</code>	Setting that keeps already-trained trees and adds new ones on the next fit
<code>set_params</code>	Method that changes a model’s settings after it is created
<code>monotonic_cst</code>	Setting that forces predictions to only rise or only fall as a column grows